Database Normalization

1NF, 2NF, 3NF, anomalies and junction tables

Watch the 4-min video

In one line: normalization stores each fact in exactly one place, which removes update, insert and delete anomalies; aim for third normal form and model many-to-many relationships with a junction table.

1 The table that drifts

A startup stores everything in one `users` table: the name, the email, the user's **roles as a comma-separated string**, and the **manager's email copied into every row**. Then a role gets renamed, a manager changes email, and the data quietly drifts out of sync. Normalization is the fix.

id	name	email	roles	manager_email
1	Asha	asha@co.io	admin,editor	ravi@co.io
2	Ben	ben@co.io	editor	ravi@co.io
3	Chen	chen@co.io	viewer,editor	ravi.k@co.io
4	Dina	dina@co.io	Editor	ravi@co.io

orange: a list in one column (breaks 1NF) · pink: copies that drifted apart ("Editor", ravi.k@co.io)

Figure 1. The starting point. Every repeated value is a copy that has to be kept in sync by hand, and some already aren't.

2 Three anomalies

Repeated data causes three classic problems:

Update anomaly

manager's email is copied into 40 rows; update 39 and one row is stale

cause: one fact, many rows

Insert anomaly

can't record a new role "auditor" until some user has it

cause: two facts, one row

Delete anomaly

delete the last admin user and the fact that the admin role exists is gone too

cause: two facts, one row

Figure 2. All three come from storing a fact in the wrong place: either many times, or only as a side effect of another fact.

3 The normal forms

First normal form (1NF): every column holds a single, **atomic** value, with no repeating groups. A comma-separated list of roles in one column breaks it: you can't index it, constrain it, or join on it cleanly (`WHERE roles LIKE '%editor%'` also matches "editor_lead").

Second normal form (2NF): every non-key column depends on the **whole** key, not just part of a composite key. It only matters when the key has more than one column.

Third normal form (3NF): non-key columns depend **only on the key**, not on other non-key columns. If `users` stores a `department_id` and a `department_name`, the name depends on the department, not the

user, so it moves to a `departments` table.

Boyce-Codd normal form (BCNF) tightens 3NF a little further (every determinant must be a candidate key). In practice, **aim for 3NF**.

> the key, the whole key, and nothing but the key

Form	Violation	Fix
1NF	`users.roles = 'admin,editor'`	One row per user–role pair in `user_roles`
2NF	Key (order_id, product_id) with `product_name`, which depends only on product_id	Move product_name to `products`
3NF	`users.department_name` depends on department_id	Move it to `departments`

4 Model entities and relationships

- **One-to-many:** a user has many orders, so `orders` gets a `user_id` foreign key. The "many" side holds the key.
- **Many-to-many:** a user can have many roles, and a role belongs to many users. Neither table can hold the other's key (that would be a list again), so you add a **junction table**.

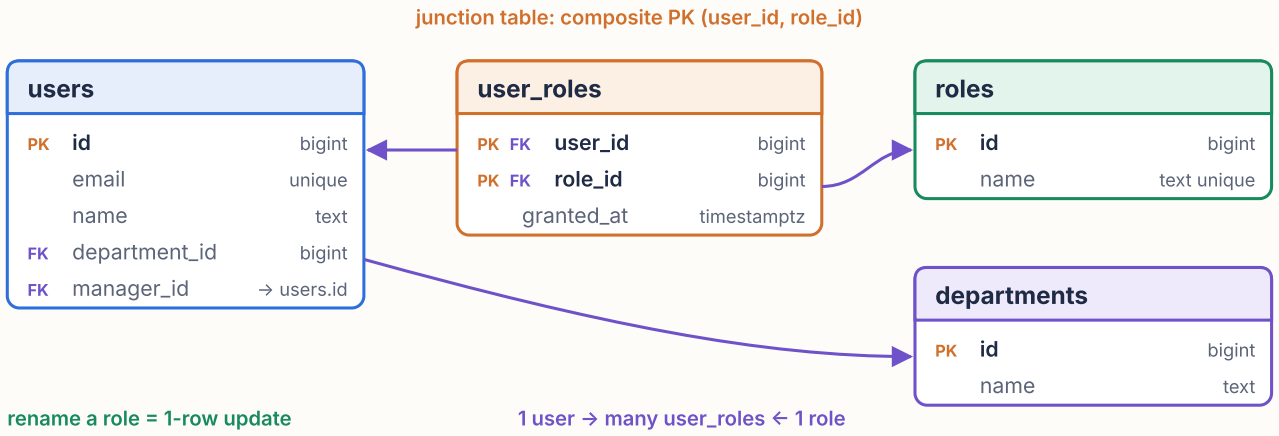


Figure 3. The normalized schema. Each fact lives once: a role's name in roles, a department's name in departments, and who has which role in user_roles.

WORKED EXAMPLE: USERS, ROLES AND USER_ROLES

`users` has an id, email and name. `roles` has an id and a **unique** name. `user_roles` has a `user_id` and a `role_id`, both foreign keys, and **together they form the primary key**, so the same role can't be assigned twice. Data about the relationship itself, like when it was granted, goes on the junction table too.

```
CREATE TABLE user_roles (
  user_id    bigint NOT NULL REFERENCES users(id),
  role_id    bigint NOT NULL REFERENCES roles(id),
  granted_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, role_id)
);

UPDATE roles SET name = 'publisher' WHERE name = 'editor';   -- 1 row, every user sees it
```

Before, renaming “editor” meant rewriting a string inside thousands of user rows. Now it is a **single-row update**. For 10,000 users averaging 2 roles each, `user_roles` holds 20,000 small rows, and “all editors” is an indexed lookup instead of a full-table `LIKE` scan.

5 Why normalize, and what it costs

Each fact lives in exactly one place, so **updates are cheap and consistent**, **constraints can guard integrity** (unique role names, valid foreign keys), and the schema stays **flexible** for questions you haven't thought of yet.

The cost: **reads need joins**. For most transactional systems, with good indexes, that's fine. When one hot read path gets too expensive, you **denormalize deliberately**, which is the next lesson.

COMMON MISTAKES

- Storing lists in a column (CSV strings, or a JSON array you need to filter or join on).
- Copying a value (manager email, department name) instead of referencing its row by key.
- A junction table with its own id but no unique constraint on (user_id, role_id), so duplicates creep in.
- Denormalizing “for speed” before measuring that the join is actually slow.

6 Key terms

Normalization: storing each fact once

Anomaly: update, insert or delete problem from duplicated data

1NF: atomic values, no repeating groups

2NF: depends on the whole composite key

3NF: depends only on the key

BCNF: stricter 3NF

Composite key: primary key made of several columns

Junction table: links two tables in a many-to-many

Denormalization: deliberate duplication for reads

7 Interview questions

Q1. Why normalize? BASIC

To store each fact once, avoiding update, insert and delete anomalies, so data stays consistent and constraints can protect it, at the cost of more joins on reads.

Q2. Model a many-to-many relationship. BASIC

Use a junction table like user_roles, with foreign keys to both sides, a composite primary key (or unique constraint) on (user_id, role_id), and any attributes of the relationship itself, like granted_at.

Q3. Explain 1NF, 2NF and 3NF in one line each. BASIC

1NF: atomic values, no lists in a column. 2NF: non-key columns depend on the whole composite key. 3NF: non-key columns depend only on the key, not on other non-key columns.

Q4. A manager's email is copied into 500 user rows and 12 are now out of date. How do you fix the design? SCENARIO

Replace manager_email with manager_id, a foreign key to users.id. The email then lives in one row; changing it is one update and all 500 reports see it through a join.

Q5. order_items has key (order_id, product_id) and also stores product_name and product_category. What is wrong? SCENARIO

A 2NF violation: name and category depend only on product_id, part of the key. Move them to products. (Keep unit_price on order_items: the price at order time is a fact about the order line.)

Q6. How do you find all users with the 'editor' role in each design? SCENARIO

Comma-separated: WHERE roles LIKE '%editor%', a full scan that also matches 'editor_lead'. Normalized: join user_roles to roles WHERE roles.name = 'editor', using an index on user_roles(role_id).

Q7. Is copying a product's price into order_items denormalization? TRICKY

No. The price at the time of the order is a different fact from today's price, and it must not change when the catalog does. Storing it is correct modeling, not duplication.

Q8. Why does user_roles need a composite primary key if each row already has a unique id column? TRICKY

A surrogate id doesn't stop the same (user_id, role_id) pair being inserted twice. A composite PK or UNIQUE (user_id, role_id) enforces “a role is assigned at most once”.

Next: SD-E-003 · Denormalization backendroadmap.com